Enunciado Arena # 1

Tree: Height of a Binary Tree

La altura de un árbol binario es el número de aristas entre la raíz del árbol y su hoja más lejana. Se te da un puntero al nodo raíz de un árbol binario. Completa la función height que retorna la altura del árbol binario.

Nota: Si el árbol tiene solo un nodo, la altura es 0.

Function Description

Completa la función height en el editor de abajo.

height tiene el siguiente parámetro:

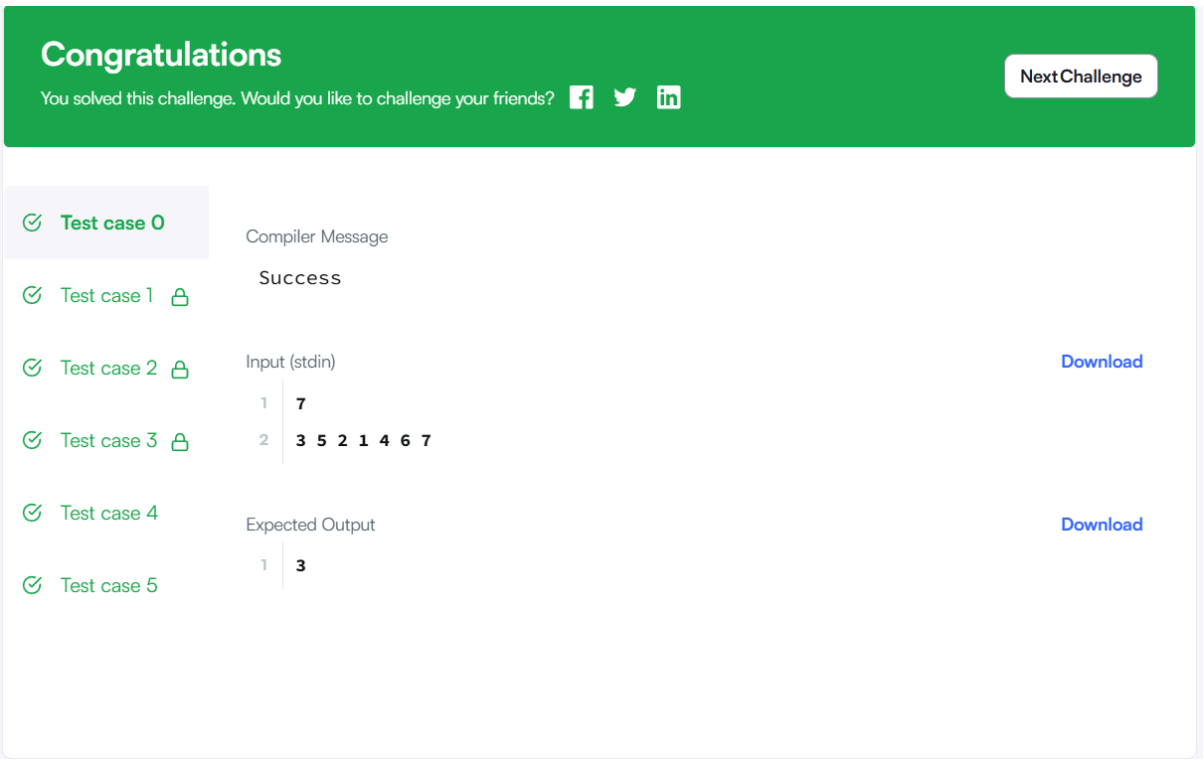
- root: un puntero a la raíz de un árbol binario.

Returns

- int: la altura del árbol.

Código:

```
def height(root):

    if root is None:

        return -1

    return 1 + max(height(root.left), height(root.right))
```



Enunciado Arena # 2

Binary Search Tree: Lowest Common Ancestor

Se te da un puntero a la raíz de un árbol binario de búsqueda y dos valores **v1** y **v2**. Debes devolver el ancestro común más bajo (LCA, por sus siglas en inglés) de **v1** y **v2** en el árbol binario de búsqueda.

Function Description

Completa la función **lca** en el editor de abajo. Debe devolver un puntero al nodo ancestro común más bajo de los dos valores dados.

lca tiene los siguientes parámetros:

- **root**: un puntero a la raíz de un árbol binario de búsqueda
- **v1**: un valor **node.data**
- **v2**: un valor **node.data**

Input Format

La primera línea contiene un entero **n**, el número de nodos del árbol.
La segunda línea contiene **n** valores separados por espacios.
La tercera línea contiene dos enteros separados por espacios, **v1** y **v2**.

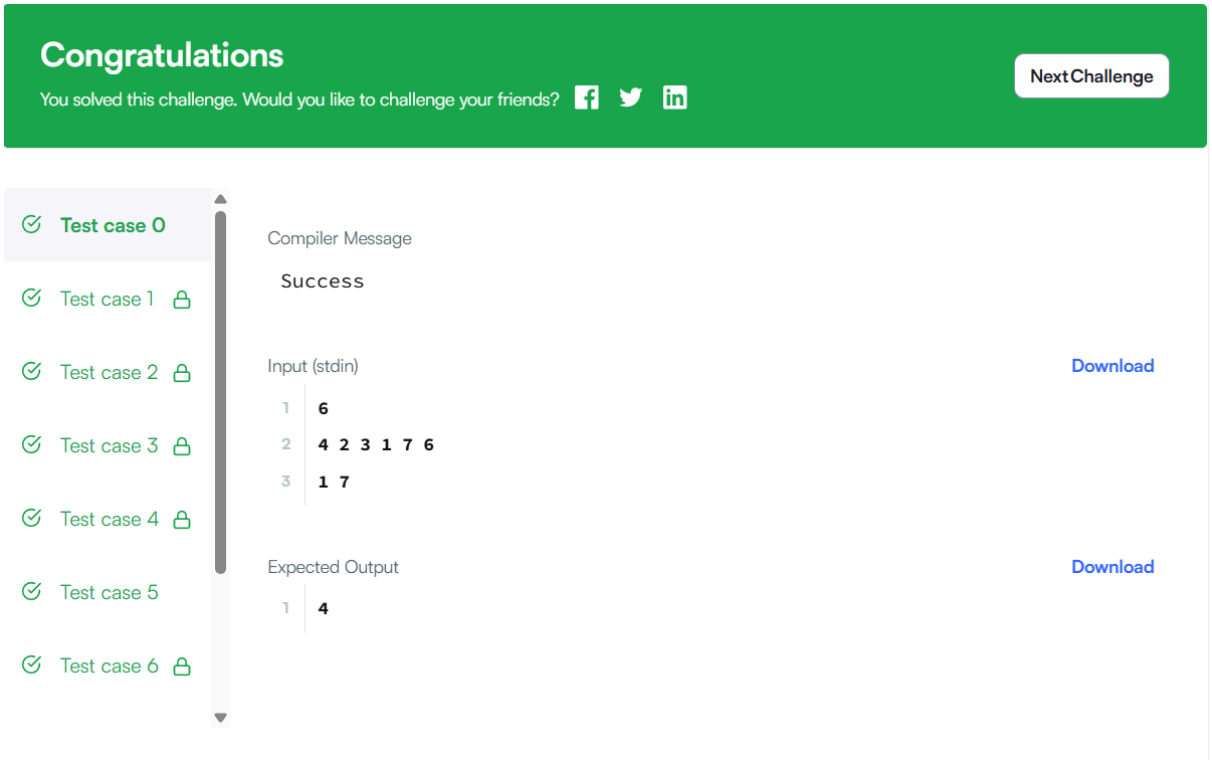
Output Format

Devuelve un puntero al nodo que es el ancestro común más bajo de **-** y **v2**.

Código:

```
class Node:
    def __init__(self, info):
        self.info = info
        self.left = None
        self.right = None
        self.level = None

    def __str__(self):
        return str(self.info)

class BinarySearchTree:
    def __init__(self):
        self.root = None

    def create(self, val):
        if self.root == None:
            self.root = Node(val)
        else:
            current = self.root

            while True:
                if val < current.info:
                    if current.left:
                        current = current.left
                    else:
                        current.left = Node(val)
                        break
                elif val > current.info:
                    if current.right:
                        current = current.right
                    else:
                        current.right = Node(val)
                        break
                else:
                    break

# Enter your code here. Read input from STDIN. Print output to STDOUT
```

```
'''
class Node:
    def __init__(self,info):
        self.info = info
        self.left = None
        self.right = None


    // this is a node of the tree , which contains info as data, left , right
'''

def lca(root, v1, v2):
    while root:
        if v1 < root.info and v2 < root.info:
            root = root.left
        elif v1 > root.info and v2 > root.info:
            root = root.right
        else:
            return root
```



Enunciado Arena # 3

Para los propósitos de este desafío, definimos un árbol binario de búsqueda como un árbol binario con las siguientes propiedades:

- El valor data de cada nodo en el subárbol izquierdo de un nodo es menor que el valor data de ese nodo.
- El valor data de cada nodo en el subárbol derecho de un nodo es mayor que el valor data de ese nodo.
- El valor data de cada nodo es distinto.

Dada la raíz de un árbol binario, ¿puedes determinar si es un árbol binario de búsqueda?

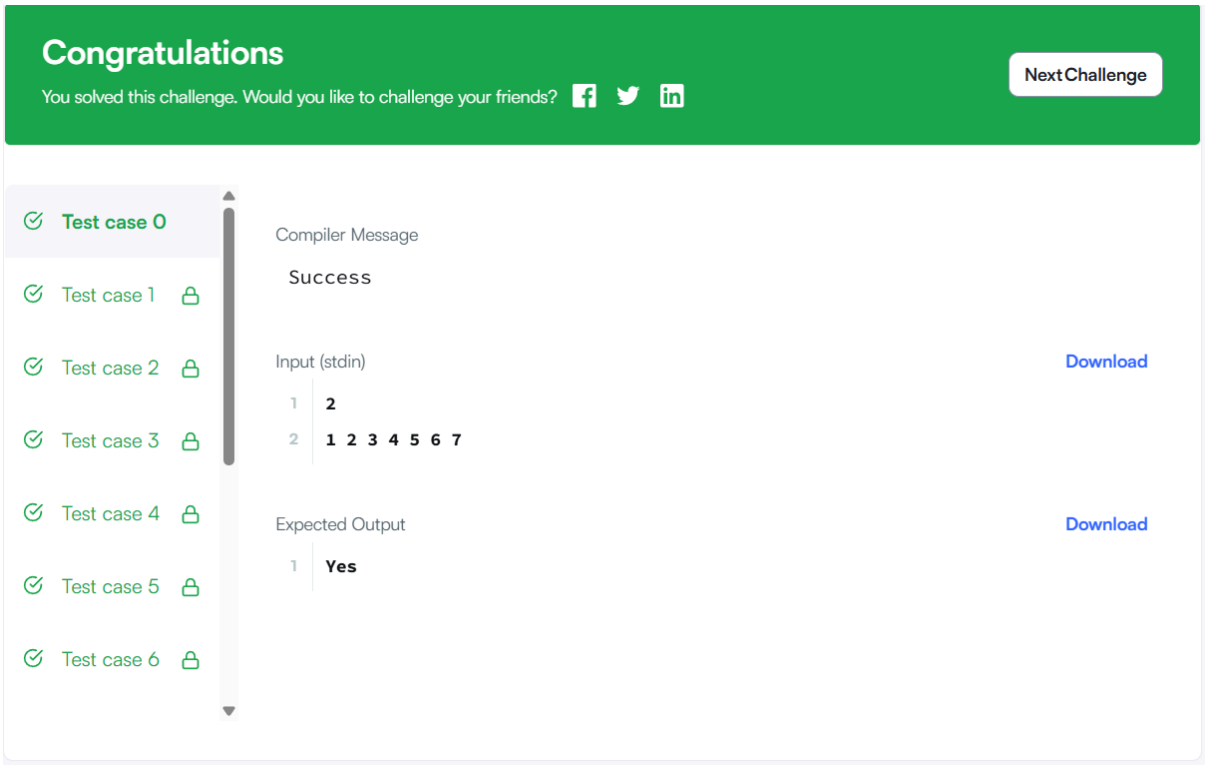
Completa la función en tu editor a continuación, la cual tiene 1 parámetro: un puntero a la raíz de un árbol binario. Debe devolver un booleano que indique si el árbol binario es o no un árbol binario de búsqueda. Es posible que debas escribir una o más funciones auxiliares para completar este desafío.

Formato de Entrada
No eres responsable de leer ninguna entrada desde stdin. Fragmentos de código ocultos ensamblarán un árbol binario y pasarán su nodo raíz a tu función como argumento.

Restricciones
0 <= data <= 10^4

Formato de Salida
No eres responsable de imprimir ninguna salida en stdout. Tu función debe devolver true si el árbol es un árbol binario de búsqueda; de lo contrario, debe devolver false. Los fragmentos de código ocultos imprimirán esta salida como un booleano.

Código:

```
def check_binary_search_tree_(root):

    valores = []



    def in_order(nodo):

        if nodo:

            in_order(nodo.left)

            valores.append(nodo.data)

            in_order(nodo.right)



    in_order(root)
```

```
for i in range(len(valores) - 1):
    if valores[i] >= valores[i+1]:
        return False

return True
```